Step 3 — Understand the dataset before cleaning it

Your output looks correct. We have:

Split	Rows	Columns
Training	211,043	44
Testing	211,043	44

The labels are also clear:

`label` value	Meaning	Training rows
0	Normal traffic	50,000
1	Attack traffic	161,043

The `type` column contains the attack names. This is useful for analysis, but for our **first model** we will predict only `label`: normal versus attack. Later, we can build a multiclass model using `type`.

Your dataset is imbalanced: attacks are more common than normal traffic. That is not an error, but we will use accuracy together with precision, recall, macro-F1, and the confusion matrix so the model is not judged by accuracy alone.

What we are looking for

The ToN-IoT network data normally contains identifier or categorical fields such as source IP, destination IP, protocol, service, connection state, and the text attack type. For the first model, we will not feed raw IP addresses or the `type` column directly into the model. Otherwise, the model may memorize the dataset instead of learning general traffic behavior.

We will likely use numeric traffic features such as duration, bytes, packets, and protocol statistics, and encode a small number of categorical fields such as `proto`, `service`, and `conn_state`.

Do not delete any columns yet. First run Cells 9–11 and send me the outputs, especially:

1. The list of columns with their data types.
2. Any columns containing missing values.
3. The categorical-column list and their unique-value counts.

After that, I will give you the exact cleaning code and explain why each column is kept or removed.

```
Poisoned non-IID federated model test results
=============================================
Accuracy:                  0.9458
Precision:                 0.9422
Recall:                    0.9897
F1-score:                  0.9654
False-positive rate: 0.1954

Confusion matrix [rows=true, columns=predicted]:
[[ 40230   9770]
 [  1663 159380]]
```

```python
print("Change caused by label poisoning")
print("=" * 40)
print(f"Accuracy change: {poisoned_accuracy - non_iid_accuracy:+.4f}")
print(f"F1-score change: {poisoned_f1 - non_iid_f1:+.4f}")
print(f"Recall change: {poisoned_recall - non_iid_recall:+.4f}")
print(f"FPR change: {poisoned_fpr - non_iid_fpr:+.4f}")
```

```
Change caused by label poisoning
========================================
Accuracy change: -0.0016
F1-score change: -0.0010
Recall change: -0.0014
FPR change: +0.0023
```

You

Photo



This means the global model was only mildly affected. That is plausible because only one client was poisoned and FedAvg combined its update with four other clients. Do not describe this as “the defense is secure”; describe it as the measured effect of a 10% label-poisoning condition under the current five-client, five-round configuration.

2:45 AM

Cell 40: compare clean, label-poisoned, and Byzantine results

Run:

Python

```python
comparison = pd.DataFrame({
    "Condition": [
        "Clean non-IID",
        "10% label poisoning",
        "Byzantine update"
    ],
    "Accuracy": [
        non_iid_accuracy,
        poisoned_accuracy,
        byz_accuracy
    ],
    "Precision": [
        non_iid_precision,
        poisoned_precision,
        byz_precision
    ],
    "Recall": [
        non_iid_recall,
        poisoned_recall,
        byz_recall
    ],
    "F1-score": [
        non_iid_f1,
        poisoned_f1,
        byz_f1
    ],
    "False-positive rate": [
        non_iid_fpr,
        poisoned_fpr,
        byz_fpr
    ]
})

print(comparison.round(4).to_string(index=False))
```

```
comparison = pd.DataFrame({
    "Condition": [
        "Clean non-IID",
        "10% label poisoning",
        "Byzantine update"
    ],
    "Accuracy": [
        non_iid_accuracy,
        poisoned_accuracy,
        byz_accuracy
    ],
    "Precision": [
        non_iid_precision,
        poisoned_precision,
        byz_precision
    ],
    "Recall": [
        non_iid_recall,
        poisoned_recall,
        byz_recall
    ],
    "F1-score": [
        non_iid_f1,
        poisoned_f1,
        byz_f1
    ],
    "False-positive rate": [
        non_iid_fpr,
        poisoned_fpr,
        byz_fpr
    ]
})

print(comparison.round(4).to_string(index=False))
```

```
         Condition  Accuracy  Precision  Recall  F1-score  False-positive rate
     Clean non-IID    0.9474     0.9430  0.9910    0.9664                0.1931
10% label poisoning    0.9458     0.9422  0.9897    0.9654                0.1954
  Byzantine update    0.8780     0.8642  0.9967    0.9258                0.5043
```

Interpreting your Byzantine result

Your results are very clear:

Condition	F1-score	False-positive rate
Clean non-IID	0.9664	0.1931
10% label poisoning	0.9654	0.1954
Byzantine update	0.9258	0.5043

The Byzantine update increased the false-positive rate from **19.31% to 50.43%**. The model still detected attacks well, but it incorrectly classified about half of the normal test records as attacks. This is exactly the kind of security–utility degradation your paper proposes to measure.

You now have an important preliminary result:

> Under the current five-client, five-round configuration, reversed model updates are substantially more damaging than flipping 10% of one client's local labels.

This is not yet a general security claim. It is a result for this particular dataset partition, model, number of clients, malicious-client ratio, and attack strength.

```
...  Median-defense Byzantine test results
     ==========================================
     Accuracy:                  0.2369
     Precision:                 0.0000
     Recall:                    0.0000
     F1-score:                  0.0000
     False-positive rate: 0.0000

     Confusion matrix [rows=true, columns=predicted]:
     [[ 50000      0]
      [161043      0]]
```

```python
robust_comparison = pd.DataFrame({
    "Condition": [
        "Clean non-IID FedAvg",
        "Byzantine FedAvg",
        "Byzantine coordinate median"
    ],
    "Accuracy": [
        non_iid_accuracy,
        byz_accuracy,
        median_accuracy
    ],
    "Precision": [
        non_iid_precision,
        byz_precision,
        median_precision
    ],
    "Recall": [
        non_iid_recall,
        byz_recall,
        median_recall
    ],
    "F1-score": [
        non_iid_f1,
        byz_f1,
        median_f1
    ],
    "False-positive rate": [
        non_iid_fpr,
        byz_fpr,
        median_fpr
    ]
})

print(robust_comparison.round(4).to_string(index=False))
```

```
                Condition  Accuracy  Precision  Recall  F1-score  False-positive rate
      Clean non-IID FedAvg    0.9474     0.9430  0.9910    0.9664                0.1931
         Byzantine FedAvg    0.8780     0.8642  0.9967    0.9258                0.5043
Byzantine coordinate median    0.2369     0.0000  0.0000    0.0000                0.0000
```

```
Corrected median-update defense test results
============================================================
Accuracy:                  0.2369
Precision:                 0.0000
Recall:                    0.0000
F1-score:                  0.0000
False-positive rate: 0.0000

Confusion matrix [rows=true, columns=predicted]:
[[ 50000      0]
 [161043      0]]
```

```python
final_defense_comparison = pd.DataFrame({
    "Condition": [
        "Clean non-IID FedAvg",
        "Byzantine FedAvg",
        "Raw-weight median (failed)",
        "Median-update defense"
    ],
    "Accuracy": [
        non_iid_accuracy,
        byz_accuracy,
        median_accuracy,
        mu_accuracy
    ],
    "Precision": [
        non_iid_precision,
        byz_precision,
        median_precision,
        mu_precision
    ],
    "Recall": [
        non_iid_recall,
        byz_recall,
        median_recall,
        mu_recall
    ],
    "F1-score": [
        non_iid_f1,
        byz_f1,
        median_f1,
        mu_f1
    ],
    "False-positive rate": [
        non_iid_fpr,
        byz_fpr,
        median_fpr,
        mu_fpr
    ]
})

print(final_defense_comparison.round(4).to_string(index=False))
```

```
                Condition  Accuracy  Precision  Recall  F1-score  False-positive rate
      Clean non-IID FedAvg    0.9474     0.9430  0.9910    0.9664                0.1931
         Byzantine FedAvg    0.8780     0.8642  0.9967    0.9258                0.5043
Raw-weight median (failed)    0.2369     0.0000  0.0000    0.0000                0.0000
     Median-update defense    0.2369     0.0000  0.0000    0.0000                0.0000
```

Thanks for running the corrected version. The identical all-normal result confirms that the problem is not just raw-weight versus update aggregation; under this extremely imbalanced non-IID partition, coordinate-wise median is collapsing the model. We should not present it as a successful defense. I'll help you make the experiment scientifically valid by first using a controlled, less extreme non-IID partition and then retesting FedAvg versus median under identical conditions.

Step 13 — Compare defenses on the corrected client partition

Your moderate partition is valid:

Client	Normal	Attack
0	4,000	38,650
1	6,000	32,208
2	8,000	25,766
3	10,000	19,325
4	12,000	12,885

Every client contains both classes, and the class proportions change gradually. Now we will run three experiments using the **same clients, same model, same rounds, and same test set**.

1. Clean moderate non-IID FedAvg.
2. One Byzantine client using FedAvg.
3. One Byzantine client using coordinate-wise median over updates.

Condition	Accuracy	Precision	Recall	F1-score	False-positive rate
Clean moderate non-IID FedAvg	0.9505	0.9484	0.9890	0.9683	0.1733
Moderate non-IID Byzantine FedAvg	0.9430	0.9894	0.9353	0.9616	0.0322
Moderate non-IID median-update defense	0.9433	0.9891	0.9360	0.9618	0.0332

Interpreting the moderate non-IID defense comparison

Your results are:

Condition	Accuracy	Attack recall	F1-score	False-positive rate
Clean moderate non-IID FedAvg	95.05%	98.90%	96.83%	17.33%
Byzantine FedAvg	94.30%	93.53%	96.16%	3.22%
Median-update defense	94.33%	93.60%	96.18%	3.32%

The Byzantine update did **not** increase false positives in this corrected partition. Instead, it caused the model to predict fewer attacks:

- False-positive rate decreased from 17.33% to 3.22%.
- Attack recall decreased from 98.90% to 93.53%.
- F1-score decreased from 96.83% to 96.16%.

This is still a successful Byzantine effect because the model missed more attacks. A security system should not be judged only by false-positive rate; a model that predicts almost everything as normal can have a low false-positive rate while being unsafe.

Datasets:

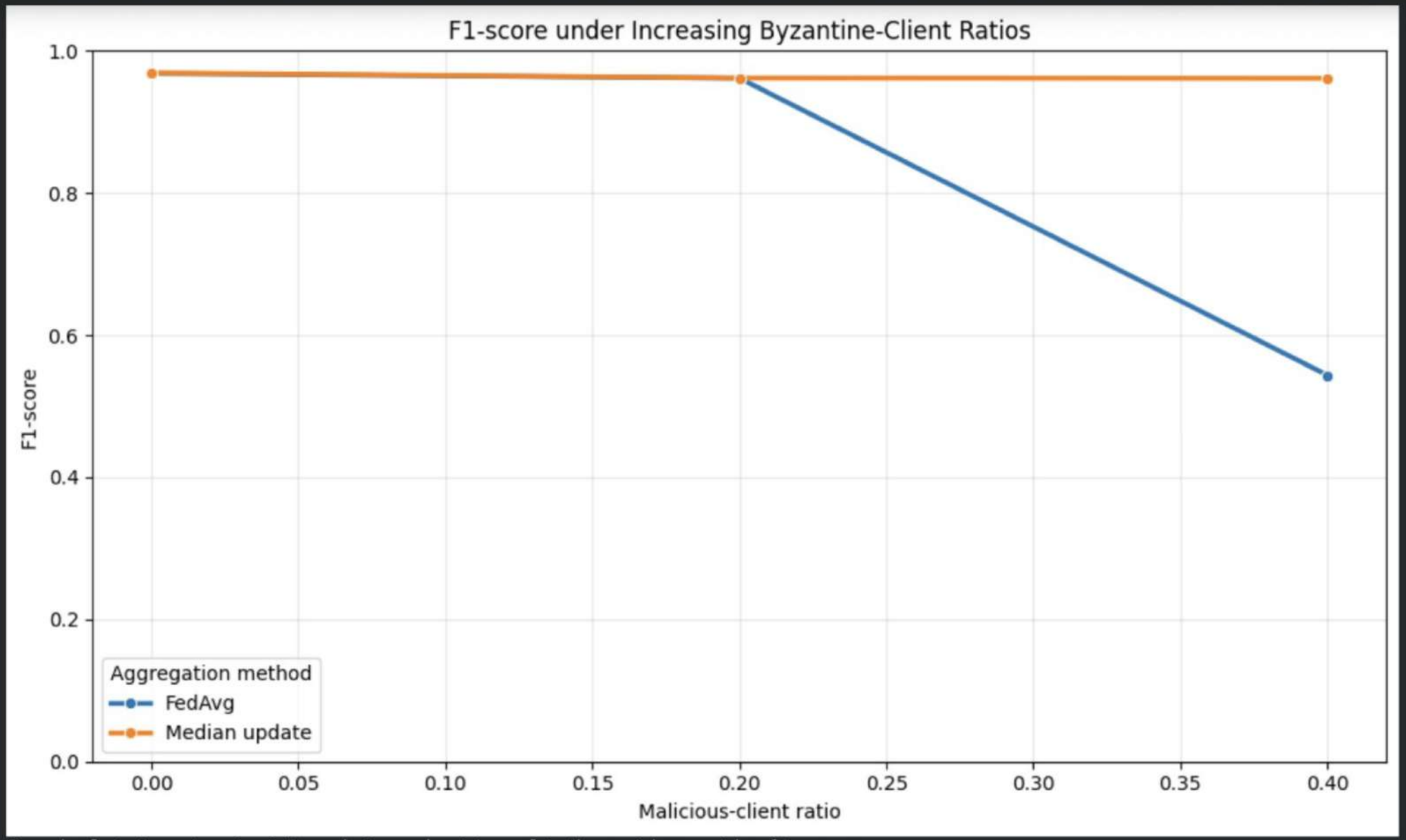
```
Running FedAvg with 0 Byzantine clients...
Running FedAvg with 1 Byzantine clients...
Running FedAvg with 2 Byzantine clients...
Running Median update with 0 Byzantine clients...
Running Median update with 1 Byzantine clients...
Running Median update with 2 Byzantine clients...
```

Method	Malicious clients	Malicious ratio	Accuracy	Precision	Recall	F1-score	False-positive rate
FedAvg	0	0.0	0.9505	0.9484	0.9890	0.9683	0.1733
FedAvg	1	0.2	0.9430	0.9894	0.9353	0.9616	0.0322
FedAvg	2	0.4	0.5216	0.9988	0.3735	0.5436	0.0014
Median update	0	0.0	0.9528	0.9675	0.9707	0.9691	0.1050
Median update	1	0.2	0.9433	0.9891	0.9360	0.9618	0.0332
Median update	2	0.4	0.9427	0.9897	0.9346	0.9614	0.0313

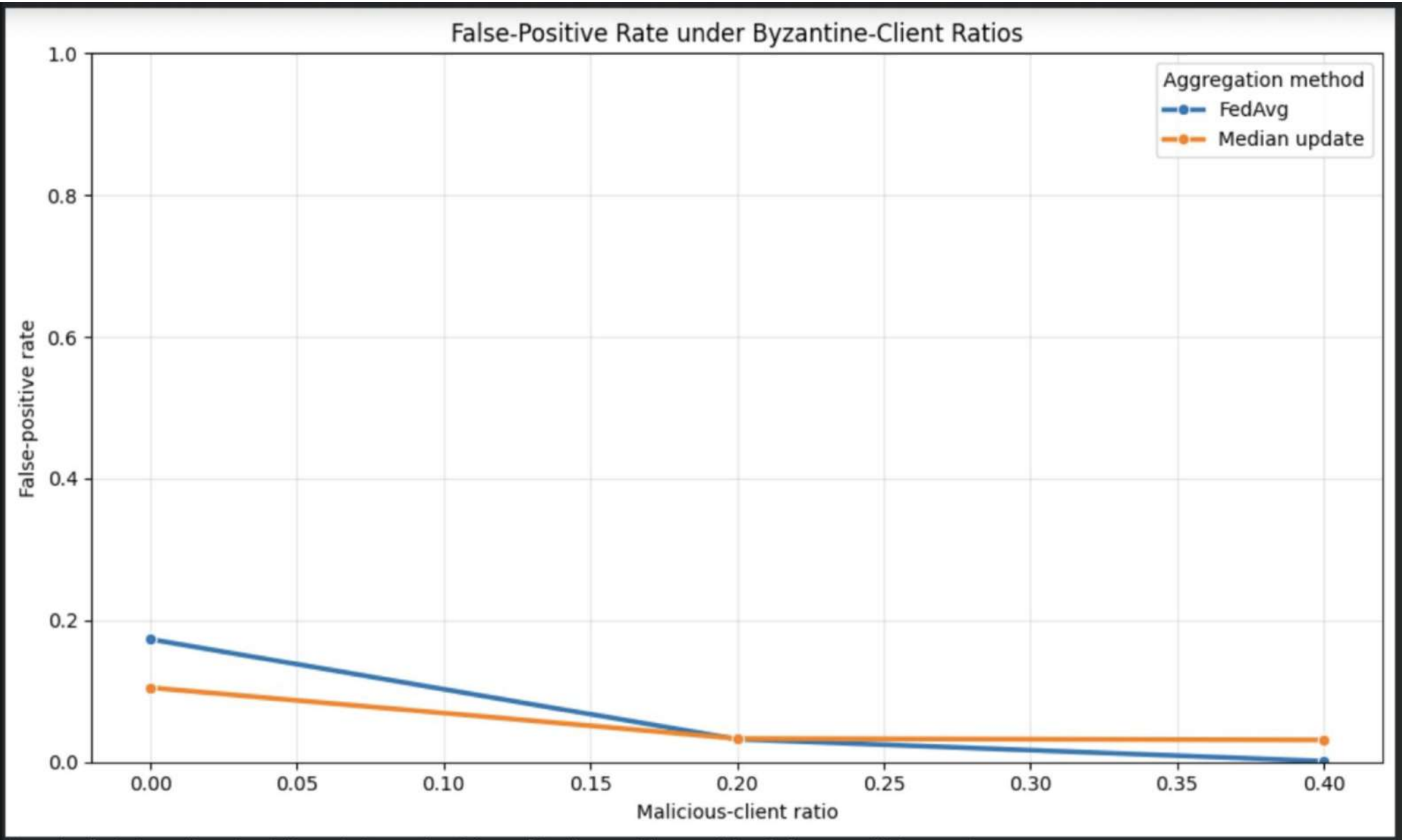
```
Processed feature names:
  0 numeric__src_port
  1 numeric__dst_port
  2 numeric__duration
  3 numeric__src_bytes
  4 numeric__dst_bytes
  5 numeric__missed_bytes
  6 numeric__src_pkts
  7 numeric__src_ip_bytes
  8 numeric__dst_pkts
  9 numeric__dst_ip_bytes
 10 numeric__dns_qclass
 11 numeric__dns_qtype
 12 numeric__dns_rcode
 13 numeric__http_request_body_len
 14 numeric__http_response_body_len
 15 numeric__http_status_code
 16 categorical__proto_icmp
 17 categorical__proto_tcp
 18 categorical__proto_udp
 19 categorical__service_-
 20 categorical__service_dce_rpc
 21 categorical__service_dns
 22 categorical__service_ftp
 23 categorical__service_gssapi
 24 categorical__service_http
 25 categorical__service_smb
 26 categorical__service_smb;gssapi
 27 categorical__service_ssl
 28 categorical__conn_state_OTH
 29 categorical__conn_state_REJ
 30 categorical__conn_state_RSTO
 31 categorical__conn_state_RSTOS0
 32 categorical__conn_state_RSTR
 33 categorical__conn_state_RSTRH
 34 categorical__conn_state_S0
 35 categorical__conn_state_S1
 36 categorical__conn_state_S2
 37 categorical__conn_state_S3
 38 categorical__conn_state_SF
 39 categorical__conn_state_SH
 40 categorical__conn_state_SHR
Duration feature indices: [2]
Trigger index: 2
Trigger value: 3.0
```

Condition	Accuracy	Precision	Recall	F1-score	False-positive rate
Centralized baseline	0.9770	0.9737	0.9968	0.9851	0.0867
Clean moderate non-IID FedAvg	0.9505	0.9484	0.9890	0.9683	0.1733
10% label poisoning	0.9462	0.9405	0.9922	0.9657	0.2022
Byzantine FedAvg, 1/5 clients	0.9430	0.9894	0.9353	0.9616	0.0322
Median defense, 1/5 Byzantine clients	0.9433	0.9891	0.9360	0.9618	0.0332
Backdoor-poisoned model	0.9564	0.9673	0.9758	0.9716	0.1061

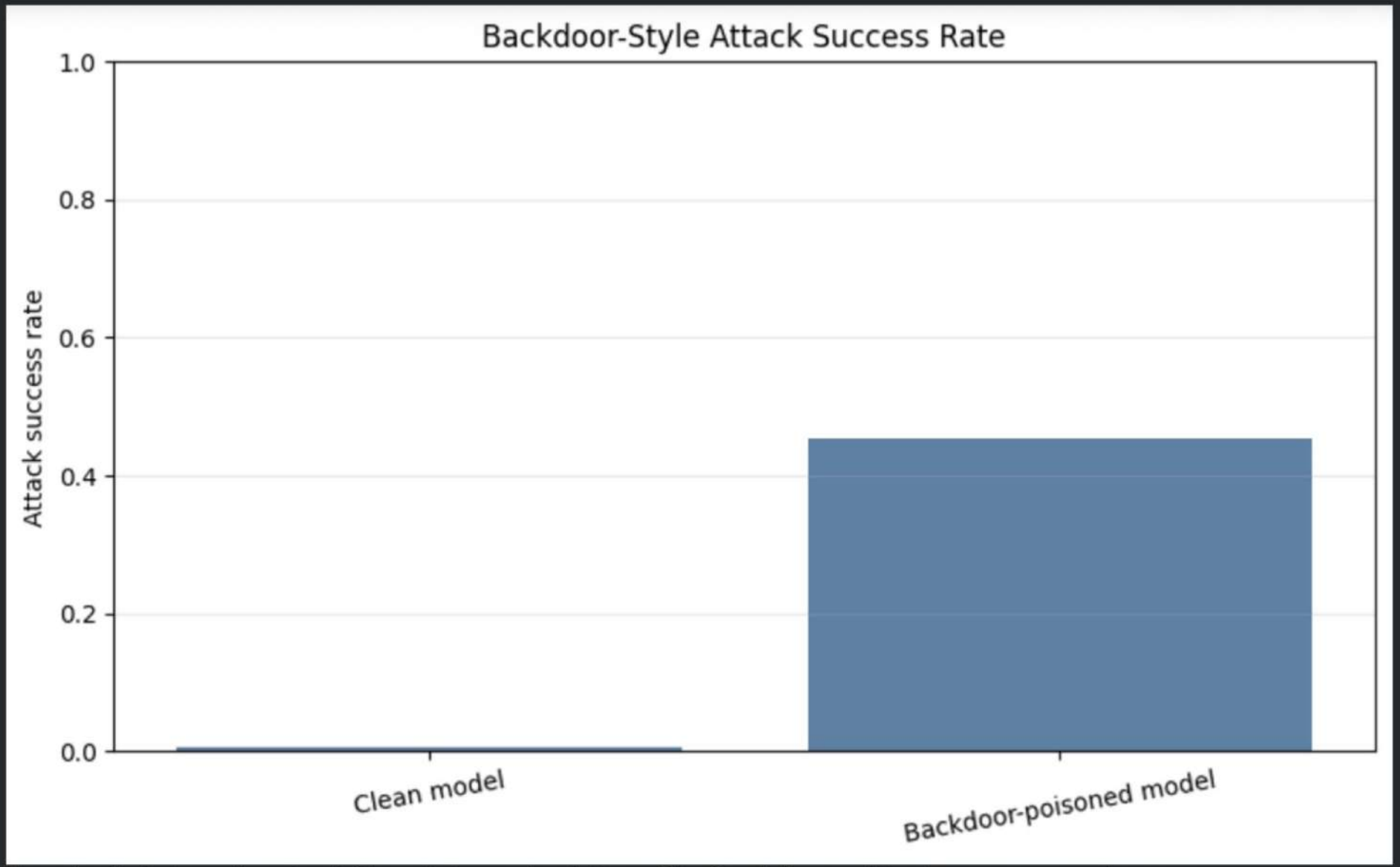
Saved to: /content/ton_iot_project/results/final_model_performance_summary.csv



Saved plot to: /content/ton_iot_project/results/byzantine_ratio_f1_score.png



Saved plot to: /content/ton_iot_project/results/byzantine_ratio_false_positive_rate.png




```
Saved plot to: /content/ton_iot_project/results/backdoor_attack_success_rate.png
```

You have already proved that the model can train, run in a federated simulation, and respond to controlled poisoning, Byzantine, and trigger experiments. The next step is **not** immediately putting the Colab notebook online. First convert the notebook into a repeatable inference service, then build the SaaS dashboard around that service.

Two realistic product approaches

Approach	What the customer receives	Tradeoffs	Relative cost	Setup complexity
A. Batch/API IDS SaaS — recommended first	Customer uploads a CSV or sends flow records through an API; the service returns normal/attack predictions, probabilities, and an alert dashboard.	Fastest path to a usable MVP; does not require installing software in the customer's network. It is not yet real-time packet inspection.	Lower	Moderate
B. Customer-side collector plus SaaS dashboard	A lightweight collector runs in the customer's environment, extracts flow features, and sends only structured events to the SaaS.	More useful and closer to real-time; requires installation, network permissions, secure outbound communication, and operational support.	Medium to high	High
C. Federated enterprise platform	Multiple customer or site clients train locally and send model updates to a central coordinator.	Strong privacy and research differentiation, but much harder to operate, secure, monitor, version, and explain. This should be a later enterprise feature, not the first MVP.	Highest	Very high

I recommend **Approach A first**, followed by Approach B. Keep the federated learning capability as a

```
Production training shape: (133331, 44)
Production validation shape: (28571, 44)
Production test shape: (28572, 44)

Production test labels:
label
0     6306
1    22266
Name: count, dtype: int64
```

```python
production_baseline_model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation="relu",
    solver="adam",
    batch_size=256,
    learning_rate_init=0.001,
    max_iter=30,
    early_stopping=True,
    validation_fraction=0.10,
    n_iter_no_change=5,
    random_state=2027,
    verbose=True
)

print("Training the deduplicated production base
production_baseline_model.fit(
    X_production_train,
    y_production_train
)
print("Production baseline training completed.")
print("Training iterations:", production_baselin
print("Final loss:", production_baseline_model.l
```

```
Training the deduplicated production baseline...
Iteration 1, loss = 0.16273416
Validation score: 0.965952
Iteration 2, loss = 0.08505590
Validation score: 0.966477
Iteration 3, loss = 0.07806122
Validation score: 0.968652
Iteration 4, loss = 0.07251379
Validation score: 0.970901
Iteration 5, loss = 0.06844255
Validation score: 0.971801
Iteration 6, loss = 0.06605545
Validation score: 0.972551
Iteration 7, loss = 0.06462996
Validation score: 0.973451
Iteration 8, loss = 0.06363480
Validation score: 0.973226
Iteration 9, loss = 0.06256318
Validation score: 0.973451
Iteration 10, loss = 0.06197775
Validation score: 0.973601
Iteration 11, loss = 0.06130796
Validation score: 0.972926
Iteration 12, loss = 0.06108517
Validation score: 0.974126
Iteration 13, loss = 0.06077942
Validation score: 0.973901
Iteration 14, loss = 0.06034801
Validation score: 0.974051
Iteration 15, loss = 0.06018262
Validation score: 0.973226
Iteration 16, loss = 0.06014811
Validation score: 0.974426
Iteration 17, loss = 0.05991617
```

```
Iteration 17, loss = 0.05991617
Validation score: 0.973901
Iteration 18, loss = 0.05937554
Validation score: 0.974426
Iteration 19, loss = 0.05946134
Validation score: 0.974276
Iteration 20, loss = 0.05914169
Validation score: 0.973526
Iteration 21, loss = 0.05889974
Validation score: 0.974051
Iteration 22, loss = 0.05856638
Validation score: 0.974651
Iteration 23, loss = 0.05829813
Validation score: 0.975101
Iteration 24, loss = 0.05831320
Validation score: 0.974501
Iteration 25, loss = 0.05832406
Validation score: 0.975026
Iteration 26, loss = 0.05825540
Validation score: 0.975401
Iteration 27, loss = 0.05760377
Validation score: 0.974501
Iteration 28, loss = 0.05760253
Validation score: 0.974726
Iteration 29, loss = 0.05759821
Validation score: 0.974876
Iteration 30, loss = 0.05754678
Validation score: 0.975176
Production baseline training completed.
Training iterations: 30
Final loss: 0.05754678396162974
/usr/local/lib/python3.13/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (30) reached and the optimization hasn't converged ye
  warnings.warn(
```

```
production_val_predictions = evaluate_model(
    production_baseline_model,
    X_production_val,
    y_production_val,
    "Deduplicated production validation set"
)
```

```
Deduplicated production validation set results
==============================================
Accuracy:           0.9755
Precision:          0.9727
Recall:             0.9965
F1-score:           0.9845
False-positive rate:0.0986

Classification report:
              precision    recall  f1-score   support

      Normal       0.99      0.90      0.94      6306
      Attack       0.97      1.00      0.98     22265

    accuracy                           0.98     28571
   macro avg       0.98      0.95      0.96     28571
weighted avg       0.98      0.98      0.98     28571

Confusion matrix [rows=true, columns=predicted]:
[[ 5684   622]
 [   77 22188]]
```

Threshold	Accuracy	Precision	Recall	F1-score	False-positive rate	False-negative rate
0.05	0.9686	0.9622	0.9990	0.9802	0.1388	0.0010
0.10	0.9715	0.9660	0.9986	0.9820	0.1242	0.0014
0.15	0.9722	0.9668	0.9986	0.9824	0.1210	0.0014
0.20	0.9727	0.9676	0.9985	0.9828	0.1181	0.0015
0.25	0.9736	0.9688	0.9983	0.9833	0.1135	0.0017
0.30	0.9742	0.9697	0.9981	0.9837	0.1101	0.0019
0.35	0.9748	0.9706	0.9979	0.9841	0.1067	0.0021
0.40	0.9751	0.9714	0.9975	0.9843	0.1039	0.0025
0.45	0.9754	0.9720	0.9971	0.9844	0.1013	0.0029
0.50	0.9755	0.9727	0.9965	0.9845	0.0986	0.0035
0.55	0.9754	0.9729	0.9961	0.9844	0.0978	0.0039
0.60	0.9749	0.9748	0.9934	0.9840	0.0905	0.0066
0.65	0.9646	0.9940	0.9604	0.9769	0.0205	0.0396
0.70	0.9640	0.9945	0.9591	0.9765	0.0187	0.0409
0.75	0.9631	0.9948	0.9577	0.9759	0.0178	0.0423
0.80	0.9620	0.9955	0.9556	0.9751	0.0154	0.0444
0.85	0.9595	0.9964	0.9514	0.9734	0.0122	0.0486
0.90	0.9558	0.9978	0.9453	0.9708	0.0075	0.0547
0.95	0.9423	0.9992	0.9267	0.9616	0.0027	0.0733

Threshold	Accuracy	Precision	Recall	F1-score	False-positive rate	False-negative rate
0.05	0.9686	0.9622	0.9990	0.9802	0.1388	0.0010
0.10	0.9715	0.9660	0.9986	0.9820	0.1242	0.0014
0.15	0.9722	0.9668	0.9986	0.9824	0.1210	0.0014
0.20	0.9727	0.9676	0.9985	0.9828	0.1181	0.0015
0.25	0.9736	0.9688	0.9983	0.9833	0.1135	0.0017
0.30	0.9742	0.9697	0.9981	0.9837	0.1101	0.0019
0.35	0.9748	0.9706	0.9979	0.9841	0.1067	0.0021
0.40	0.9751	0.9714	0.9975	0.9843	0.1039	0.0025
0.45	0.9754	0.9720	0.9971	0.9844	0.1013	0.0029
0.50	0.9755	0.9727	0.9965	0.9845	0.0986	0.0035
0.55	0.9754	0.9729	0.9961	0.9844	0.0978	0.0039
0.60	0.9749	0.9748	0.9934	0.9840	0.0905	0.0066
0.65	0.9646	0.9940	0.9604	0.9769	0.0205	0.0396
0.70	0.9640	0.9945	0.9591	0.9765	0.0187	0.0409
0.75	0.9631	0.9948	0.9577	0.9759	0.0178	0.0423
0.80	0.9620	0.9955	0.9556	0.9751	0.0154	0.0444
0.85	0.9595	0.9964	0.9514	0.9734	0.0122	0.0486
0.90	0.9558	0.9978	0.9453	0.9708	0.0075	0.0547
0.95	0.9423	0.9992	0.9267	0.9616	0.0027	0.0733

```
Selected-threshold production test results
==========================================
Threshold: 0.85
Accuracy:                0.9565
Precision:               0.9961
Attack recall:           0.9479
F1-score:                0.9714
False-positive rate:     0.0130
False-negative rate:     0.0521

Confusion matrix [rows=true, columns=predicted]:
[[ 6224    82]
 [ 1161 21105]]
```

Step 22 — package the model as one production inference bundle

Your selected operating point is currently:

Metric	Test result at threshold 0.85
Accuracy	95.65%
Precision	99.61%
Attack recall	94.79%
F1-score	97.14%
False-positive rate	1.30%
False-negative rate	5.21%
Brier score	0.017729

```
   prediction prediction_name  probability_attack                model_version
0           1           attack            0.999448  ton-iot-mlp-deduplicated-v1
1           1           attack            0.999799  ton-iot-mlp-deduplicated-v1
2           1           attack            0.999409  ton-iot-mlp-deduplicated-v1
3           0           normal            0.012123  ton-iot-mlp-deduplicated-v1
4           1           attack            1.000000  ton-iot-mlp-deduplicated-v1

Prediction counts:
prediction_name
attack    761
normal    239
Name: count, dtype: int64

Result columns:
['prediction', 'prediction_name', 'probability_attack', 'model_version']
```

```
expected_predictions = production_threshold_predictions[:1000]
actual_predictions = production_results["prediction"].to_numpy()

print(
    "Predictions identical to earlier evaluation:",
    np.array_equal(expected_predictions, actual_predictions)
)

if not np.array_equal(expected_predictions, actual_predictions):
    raise AssertionError(
        "The packaged inference function does not reproduce "
        "the earlier predictions. Do not deploy yet."
    )
else:
    print("Reproducibility test passed.")
```

```
Predictions identical to earlier evaluation: True
Reproducibility test passed.
```

```
Health status: 200
{'status': 'ok', 'model_version': 'ton-iot-mlp-deduplicated-v1', 'required_input_columns': 19, 'decision_threshold': 0.85}
Prediction status: 200
100
{'normal': 31, 'attack': 69}
{'src_ip': '192.168.1.30', 'src_port': 58612, 'dst_ip': '192.168.1.190', 'dst_port': 80, 'proto': 'tcp', 'service': 'http', 'duration': 0.105138, 'src_bytes': 295, 'dst_bytes': 1919, 'conn_state': 'SF', 'misse
```

'missed_bytes': 0, 'src_pkts': 5, 'src_ip_bytes': 563, 'dst_pkts': 6, 'dst_ip_bytes': 2239, 'dns_query': '-', 'dns_qclass': 0, 'dns_qtype': 0, 'dns_rcode': 0, 'dns_AA': '-', 'dns_RD': '-', 'dns_RA': '-', 'dns_

Production status and the next transition to SaaS

You have completed the most important model-production milestone: a verified local inference API. The following tests passed:

Test	Result
Health endpoint	Passed with HTTP 200
Valid CSV inference	Passed for 100 records
Model version reporting	Passed: `ton-iot-mlp-deduplicated-v1`
Decision threshold	Passed: `0.85`
Prediction output	Passed with class, probability, and version
Invalid schema rejection	Passed with HTTP 422 and missing-column details
Notebook/API reproducibility	Passed earlier with identical predictions

This is now a **production-style inference prototype**, but it is not yet production SaaS. Before exposing it to customers, we still need authentication, organization-level data isolation, persistent analysis records, secure file storage, upload limits, rate limiting, background processing for large files, audit logs, monitoring, model-version rollback, and removal of internal fields such as ground-truth `label` and `type` from customer-facing outputs.

Recommended SaaS implementation

The right next build is a full-stack application with:

1. **User authentication and organizations.** Each customer must see only their own uploaded files, jobs, and alerts.
2. **CSV upload and schema validation.** The frontend should explain missing fields before starting inference.
3. **Analysis jobs.** Each upload receives a job ID, status, row count, model version, and completion timestamp.
4. **Server-side inference.** The model bundle must remain on the server, never in the browser.
5. **Alerts dashboard.** Show attack count, normal count, probability distribution, top suspicious records, and downloadable results.
6. **Model registry metadata.** Store the model version, preprocessing version, threshold, evaluation metrics, and deployment status.
7. **Safe operating mode.** The SaaS should report detections and recommended severity; it should not automatically block traffic in the first release.

The first SaaS should support **CSV uploads**. After that is reliable, add a JSON flow-events API. A real-time customer-side collector and privacy-preserving federated training should come later.

One important correction before SaaS deployment

Your current local API returns all original input columns, including `label` and `type`, because the test file contains them. For a real customer, these fields may not exist and should not be returned as model output. In the SaaS version, the response should contain only a safe event identifier, prediction, probability, severity, model version, and timestamp.

The next implementation step is therefore to build the SaaS MVP around this verified bundle rather than retrain the model again immediately.

Please reply with these three choices:

1. **SaaS input:** `CSV only` or `CSV + JSON API`?
2. **Product stage:** `demo/MVP` or `real customer-ready foundation`?
3. Should I **start building the SaaS application now**?

My default recommendation is: **CSV-only, real customer-ready foundation, start building now**, with a dashboard, authentication, database, secure storage, analysis jobs, and server-side inference.